DECIDING WITHOUT GENERATING: WHEN DOES A STANDALONE DECISION MODEL PAY OFF IN LLM SYSTEMS?

Anonymous Authors[1]

ABSTRACT

Much of what a language model does inside a production system is not writing but deciding: which model should answer, whether a cached answer can be reused, which tool to call next. A new class of non-generative "decision models" returns a typed, calibrated choice instead of text and is marketed as a far cheaper replacement for an LLM at these points. We test that claim on one harness with interchangeable deciders (a frontier LLM, small hosted and self-hosted LLMs, a proprietary decision model, an open-weights reproduction of it, and trained classifiers) at three decision points, measuring quality, dollars, latency and GPU energy. The decision model pays off as a semantic-cache guard: on near-identical questions with different answers it decides correctly 93.3% of the time where a similarity threshold reaches 66.1%, at about 2% of the frontier model's price per decision. It does not pay off for agent tool choice, where letting the generator decide inside its own call is cheapest at equal task reward, or for model routing, where a small classifier trained on the cheap model's record beats every general decider. A separate decider is therefore worth its call when the decision avoids a generation and the evidence it needs is in the input. We also show that the standard semantic-caching benchmarks carry label errors that come from how they were built, and we release corrected labels.

1 INTRODUCTION

A deployed LLM system makes many small decisions around each answer it writes. A router decides which model should handle a prompt (Chen et al., 2023; Ong et al., 2024). A semantic cache decides whether a stored answer can be served instead of a new one (Bang, 2023; Schroeder et al., 2025). An agent decides which tool to call next, or whether to stop and reply (Yao et al., 2024). Each of these has a small, fixed set of outcomes, yet the usual way to make them is to ask a generative model and parse its text, paying for a full autoregressive call to obtain a few bits.

A recent class of models targets exactly this gap. These *decision models*, which their vendors call "System One" models after the fast mode of dual-process theory (Kahneman, 2011), are not autoregressive: given a state and a typed question (pick one of these options, score on this scale, yes or no), they return the answer with a probability for each option in a single pass. The first commercial one, Jev (TypeSafe AI, 2026), is priced at a small fraction of a frontier LLM per input token with no output cost, and is advertised as a drop-in replacement for an LLM wherever the output is a decision. An open-weights reproduction, Kev (Palmer, 2026), followed within weeks.

Whether this replacement pays is an empirical systems question, and it has two parts that vendor numbers do not separate. The first is quality: is the cheap decision as good as the expensive one? The second is structure: what does the decision buy? A cache guard's decision can remove a generation entirely, so a cheap, accurate guard saves nearly the whole call. An agent's tool choice sits in front of a generation that happens anyway, because some model must still write the tool's arguments, so a separate decider adds a call instead of removing one.

We study both parts on one harness. Every decider implements the same interface (a state, a set of options, a returned choice with a confidence), so the same code path scores a frontier LLM, a small hosted LLM, a self-hosted LLM, Jev, Kev, and trained classifiers. We run them at three decision points on public benchmarks, with repeated calls, clustered confidence intervals, measured dollar cost per call, and measured GPU energy for the self-hosted deciders.

Our findings are as follows.

Cache guard: the decision model pays off. On GSM-Plus (Li et al., 2024), where a question and its nearest cached neighbour often differ by one number, embedding similarity cannot tell a safe reuse from a wrong one, and a dev-tuned threshold reuses almost nothing (66.1% accu-

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the Machine Learning and Systems (MLSys) Conference. Do not distribute.

racy). Jev reaches 93.3%, within 2.6 points of the frontier model, at about 2% of its price per decision. A frontier model used as the guard is self-defeating: its decision costs more than the answer it might save. On two realistic traffic benchmarks Jev ties the threshold where similarity works and beats it by 4.4 points where it does not, and a cascade that calls Jev only inside the ambiguous similarity band is best or tied for best on all three datasets.

Agent control: it does not. On τ^2-bench airline tasks (Barres et al., 2025), Jev alone picks the reference action 81.4% of the time at $1/22$ of the frontier model's cost and latency. Inside a real agent loop, however, the arm in which the generator decides and acts in one call is the cheapest of all and matches the best reward. Splitting the decision out either costs more (the generator asked twice) or loses about five points of reward (Jev deciding).

Routing: a trained classifier wins. On RouterBench (Hu et al., 2024), no general decider, frontier LLM included, routes usefully from the prompt alone. A small classifier trained on the cheap model's recorded correctness beats all of them, and even it recovers little.

The pattern. A standalone decider is worth its call when two conditions hold: the decision avoids a generation, and the evidence it needs is visible in its input. Cache guarding meets both. Agent control fails the first and routing fails the second.

A benchmark flaw. While running the cache guard we found that every decider, the frontier model included, stopped at about 71% accuracy on the two standard semantic-caching benchmarks (Schroeder et al., 2025). The cause is the labels: they are inferred from classes (transitive closures of pairwise LLM judgements, or "generated from the same seed") and not judged on the pair a cache actually meets. We relabel every pair we use under a written guideline, changing 6.0% and 15.7% of labels, and release the corrections with the reasons for each.

We make three contributions: (1) an independent measurement of the decision-model paradigm at three LLM decision points, in dollars, seconds and joules, against open baselines that run without any API; (2) a decider interface and harness that make a typed-output decision model interchangeable with chat LLMs and classifiers; and (3) an analysis of how class-based labelling breaks semantic-cache benchmarks, with corrected labels.

2 Background

Decision models. A decision model takes a *state* (any text) and one or more typed *questions*. The question type we use is a choice: a set of named options, each with a short description, plus instructions. The model returns the chosen option and a probability for every option. Nothing is generated token by token, so latency and cost depend on the input length only.

Jev (TypeSafe AI, 2026) is proprietary and reachable only through an API; its weights, architecture and training data are not public. We pin version 1.13 and treat every vendor claim about it as a hypothesis to test. Kev (Palmer, 2026) is a third-party, Apache-2.0 reproduction that speaks the same wire format, in 0.8B, 4B and 9B sizes. Kev is not Jev's weights: its results describe an open decision model run locally, not Jev itself. We include it for two reasons: it lets every experiment run without an API key, and it lets us measure model-only latency and GPU energy, which a hosted API hides.

Three decision points. Table 1 lists the three decision points we study and how they differ. What matters for the economics is the column "decision replaces generation". In routing and cache guarding, one outcome of the decision skips an expensive call. In agent control, a generative model must run after the decision whatever it is, because tool arguments and user-facing replies are free text.

3 Harness and Method

3.1 One interface, interchangeable deciders

A decider receives a rendered state, an instruction, and a set of options, and returns one option with a confidence. Every decider at a decision point sees the identical rendered state and the identical option descriptions. We evaluate the following deciders.

Decision models. Jev 1.13 through its hosted API; Kev (4B by default, also 0.8B and 9B for agent control) served locally.

LLMs. GPT-5.6 as the frontier ceiling; GPT-OSS-20B as a small hosted LLM; Qwen3-8B (Yang et al., 2025) served locally with vLLM (Kwon et al., 2023), with thinking turned off. An LLM decides through one constrained call: the options are the only permitted outputs (a forced tool choice), and its confidence is the one it states.

Baselines specific to a decision point. A similarity threshold, an off-the-shelf duplicate-question cross-encoder (Reimers & Gurevych, 2019), the vCache method (Schroeder et al., 2025), and a classifier fine-tuned on a dev split (cache guard); a classifier fine-tuned on the cheap model's recorded correctness, plus always-small, always-large and random-mix policies (router).

Cascades. A cheap first stage answers when it is confident and escalates otherwise: Jev to GPT-5.6 (agent control), and similarity threshold to Jev (cache guard). Every cascade threshold or band is chosen on data the reported number

Table 1. The three decision points. "Cheap alternative" is the baseline a decider must beat to justify its own call.

Decision point	Options	Benchmark (test size)	Replaces generation?	Cheap alternative
Agent control	next tool, or reply	τ^2-bench airline (1,076 states; 50 tasks)	No	generator's own call
Cache guard	reuse / regenerate	GSM-Plus, LMArena, SearchQueries (2,000 each)	Yes	similarity threshold
Router	small / large model	RouterBench (1,998 prompts)	Yes	trained classifier

does not use.

3.2 Measurement

Repeats and intervals. Stochastic deciders are called five times per item (GPT-5.6 three times at the cache guard and router, to bound cost). Repeats on one item are correlated, so all intervals are 95% cluster bootstraps: over states or tasks for agent control, over classes for the cache guard, over prompts for the router. Comparisons between two deciders are paired on the same items.

Dollars. For hosted models we record the cost the API gateway reports for each call, not a token count multiplied by a list price; the two differed by about $3\times$ for one model because of upstream provider selection. All hosted calls go through one gateway and one key. Calls from different deciders are interleaved so that network conditions affect them equally.

Latency. Wall-clock time per call at the client. For local deciders we also record server-side model time, which isolates HTTP overhead (7–8 ms).

Energy. For local deciders we read the GPU's cumulative energy counter (NVML) around a contiguous block of calls on a dedicated A100-80GB, with one request in flight, prefix caching off, and warm-up calls discarded. We report gross joules per decision and the value net of the idle baseline. The counter is validated against the instantaneous power reading before each run; on one consumer GPU it failed that check, and we report no energy for that machine. Hosted deciders have no energy number.

Failures. A call that fails for an infrastructure reason (billing, rate limit, provider error, network) is retried and never counted against the decider. A call in which the decider produces no valid choice is a decider failure, counted as wrong and never retried, so a stochastic decider cannot re-roll its own failures.

Calibration. Expected calibration error (ECE) (Guo et al., 2017) over the probability of the chosen option.

4 Agent Control

Task. At each agent turn, choose the next tool or choose to reply to the user. We use the airline domain of τ^2-bench (Barres et al., 2025) (50 tasks, a simulated user, real tools, and the benchmark's own grading), pinned to release v1.0.1.

Table 2. Decision-only replay on 1,076 airline states, 5 repeats. Accuracy = agreement with the reference action. †Graded against its own trajectories, so an upper bound.

Decider	Acc. (%)	p50 (ms)	$/decision	ECE
GPT-5.6†	88.5	3,909	3.83×10^{-3}	0.103
Jev → GPT-5.6	84.9	162	8.16×10^{-4}	0.063
Jev	81.4	176	1.73×10^{-4}	0.029
GPT-OSS-20B	67.0	1,299	3.11×10^{-4}	0.286
Kev-9B	40.1	548	local	0.179
Qwen3-8B	39.6	656	local	0.599
Kev-4B	32.1	439	local	0.124
Kev-0.8B	27.3	218	local	0.088

We run two experiments. The first scores the decision in isolation; the second places each decider inside the agent loop.

4.1 Decision-only replay

We collect agent states from GPT-5.6 acting as a normal single-call agent (50 tasks $\times$ 3 episodes) and keep the 1,076 states from episodes that succeeded. Each decider is then asked, at each saved state, for the next action; nothing is executed. The label is the action the reference agent took. This is a lower bound on quality, since other valid actions exist, and it favours GPT-5.6, which is graded against its own behaviour.

Table 2 shows the result. Jev agrees with the reference on 81.4% of states [79.1, 83.6], 7.1 points below GPT-5.6 [5.2, 9.1], while being 22 times cheaper and 22 times faster at the median, and the best calibrated decider (ECE 0.029). It is 14 points more accurate than the small hosted LLM at about half its cost. The vendor's efficiency claim therefore holds for the single decision, at a measured quality cost.

A cascade that escalates to GPT-5.6 when Jev's confidence is below 0.65 (a threshold chosen on held-out tasks) recovers half of the gap: 84.9% [82.8, 86.9], with 16.7% of calls escalated, at 21% of GPT-5.6's cost and Jev's median latency.

The open deciders are far behind on this domain. Kev reaches 27–40% depending on size and Qwen3-8B 39.6%. Kev also rejects 34 states (3%) that exceed a hard input limit

Table 3. Airline tasks in the agent loop (G = GPT-5.6, 250 episodes per arm). Reward with 95% CI over tasks; pass^5 = all five trials of a task succeed; decision share = decider's part of the agent's cost.

Arm	Reward	pass^5	$/success	Dec. share
A: Jev + G	.744 [.636, .844]	0.600	0.048	5.3%
B: G + G	.796 [.696, .888]	0.660	0.176	74.5%
C: G alone	.792 [.696, .884]	0.640	0.033	0%
D: GPT-OSS + G	.692 [.572, .804]	0.540	0.043	4.9%
E: Kev-4B + G	.045 [.012, .089]	–	0.476	0%

in its encoder; we count these as failures. Qwen3-8B is almost always certain (mean confidence 0.996) and often wrong.

Energy. On the same A100-80GB, a decision costs 47 J with Kev-0.8B, 132 J with Kev-4B, 170 J with Kev-9B and 211 J with Qwen3-8B (gross; 29, 100, 130 and 167 J net of idle). At similar accuracy (Kev-9B versus Qwen3-8B), the decision model uses about 20% less energy and is 17% faster; the gap is real but far from the order-of-magnitude difference seen in API prices.

4.2 In the agent loop

A decision model cannot write tool arguments or a reply, so in a real agent a generator G must act after every decision. We compare five arms on the same tasks, with the same user simulator and seeds, and G = GPT-5.6:

A: Jev decides, then G executes the chosen action. **B**: G decides in a constrained call, then G executes. **C**: G decides and executes in a single call, the standard agent. **D**: GPT-OSS-20B decides, then G executes. **E**: Kev-4B decides, then G executes.

If a decider fails on a turn, G decides that turn and we report the fallback rate. Each arm runs 50 tasks $\times$ 5 trials.

Table 3 and Figure 1 show that the single-call agent (C) is the cheapest arm and ties for the best reward. Asking G for the decision separately (B) changes reward by $+0.4$ points $[-2.8, +4.0]$ and adds \$0.114 per episode, because the long conversation is now sent twice per turn. Replacing that decision with Jev (A) removes almost all of the added cost, so A is \$0.105 per episode cheaper than B, but A still costs \$0.009 more per episode than C and loses 4.8 points of reward against it $[-10.8, +0.8]$, and 5.2 points against B $[-10.8, -0.4]$. The single-decision gap of Table 2 carries into the loop almost unchanged.

Arm E fails: Kev-4B asks for user details before the user has given an identifier, and repeats the call until the episode is cut off. The same behaviour appears with a local Qwen3-8B as G (reward 0.048), so it belongs to the decider and not to the generator.

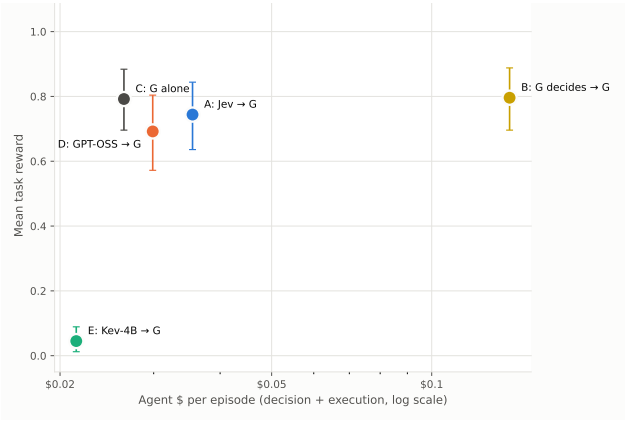


Figure 1. Task reward against agent cost per episode for the five arms (log cost axis; error bars are 95% CIs over tasks). The single-call agent C is cheapest at the top reward.

One caveat runs the other way. With a weak generator (G = Qwen3-8B, all local), separating the decision appears to help: arm B reaches 0.308 [0.196, 0.428] against 0.196 [0.116, 0.284] for C. A split may help a small model structure its work; it does not help a frontier one.

Takeaway. When a generation follows every decision, the decision is nearly free inside that generation, and a separate decider can only add cost or lose quality.

5 Cache Guard

Task. A semantic cache has found the stored prompt nearest to a new query. Should it *reuse* the stored answer or *regenerate*? A correct reuse skips the model call; a wrong reuse serves a wrong answer.

Scope of the decision. Deciders see the new query and the cached query, and never the cached answer or the similarity score. The decision is "are these the same request?". Whether the stored answer was right belongs to the model that wrote it and to the cache's expiry policy, and reading long answers would cost much of what the cache saves. An earlier version of our question asked deciders to check the stored answer; the frontier model then regenerated on 24% of pairs whose two prompts were *identical*, because it disagreed with the stored answer. We discarded those runs.

5.1 Data

We use three public sets and build nothing ourselves, so that we cannot choose the hard cases. Each is a stream: the cache holds every earlier prompt, and each query's candidate is its nearest cached neighbour under GTE-large embeddings (Li et al., 2023), the same embedding for every method. Each set is split by class into 600 dev queries (for tuning thresh-

olds, bands and the trained classifier) and 2,000 test queries.

GSM-Plus (Li et al., 2024): perturbed variants of GSM8K questions (Cobbe et al., 2021), with the original questions as the cache. Rephrasings keep the answer and number changes do not, so reuse is correct for 33.6% of queries. Labels are exact numeric answers. Here wrong reuses are *more* similar to their neighbour than correct ones (median cosine 0.966 against 0.955), so no threshold can separate them.

LMArena and **SearchQueries** (Schroeder et al., 2025): the standard semantic-caching benchmarks, built from chat prompts (Chiang et al., 2024) and real search queries (Craswell et al., 2020). Reuse is correct for 92.5% and 58.2% of queries after the label correction of Section 5.4.

5.2 Results

Table 4 and Figure 2 give the results.

Where similarity is blind, the decision model wins by a wide margin. On GSM-Plus the dev-tuned threshold, the cross-encoder and vCache all reuse almost nothing, because no cut-off gives acceptable precision; their accuracy is the regenerate-always rate. vCache keeps its error bound on every dataset, and pays for it on the two traffic sets by regenerating on 45–48% of queries that could have been reused. Jev reaches 93.3%, which is 27.2 points above the threshold [25.5, 29.1] and 11.7 points above a classifier trained on this dataset's own dev split [10.1, 13.3], with a 1.1% wrong-reuse rate. GPT-5.6 is 2.6 points better [1.9, 3.4] at 48 times the price per decision.

Where similarity works, the decision model ties it. On LMArena the threshold and Jev are statistically tied on accuracy ($+0.6$ for the threshold $[-0.4, +1.6]$), and Jev serves fewer wrong answers (-1.7 points $[-2.3, -1.2]$). On SearchQueries Jev is 4.4 points more accurate [2.3, 6.4] and cuts wrong reuses from 14.4% to 3.8%. GPT-5.6 is not better than Jev on either set; on LMArena it is the weakest LLM decider, because it regenerates on 8.4% of queries that could have been reused.

A cascade gets the best of both. Calling Jev only when the similarity falls in an ambiguous band chosen on dev (3% of LMArena queries, 72% of SearchQueries, all of GSM-Plus) is best or tied for best on every dataset, and on LMArena costs 6×10^{-7} dollars per query.

The open decision model is competitive here. Unlike in agent control, Kev-4B is within a point of Jev on the two traffic sets and 5.2 points behind on GSM-Plus, at 70 ms per decision on a local GPU, though it serves 1.6 to 5 times as many wrong answers.

5.3 What the decision is worth

A guard only pays if its decision is much cheaper than the answer it saves. We measured the cost of regenerating with GPT-5.6 on 200 queries per dataset: \$0.0016 on GSM-Plus and \$0.0084 on LMArena. The expected cost of a query is then the decision's cost plus the regeneration cost times the share of queries regenerated.

On GSM-Plus, where only a third of queries can be reused, Jev as the guard brings the cost to 72% of running without a cache; the similarity methods stay at about 100%. GPT-5.6 as the guard raises it to 138%: the decision alone costs \$0.0011, two thirds of the answer it is deciding about. On LMArena, where most queries can be reused, the free threshold brings the cost to 5% of no-cache, the cascade to 5%, Jev on every query to 9%, and GPT-5.6 to 28%. These figures do not price a wrong answer; with any positive price for one, the comparison on LMArena and SearchQueries moves further toward Jev and the cascade, which serve fewer.

5.4 Label errors in semantic-caching benchmarks

On our first run, every decider reached only 70–72% on LMArena and SearchQueries, the frontier model included, while the same deciders reached 93–97% on GSM-Plus. A frontier model failing "is this the same request?" on short queries pointed at the labels. Reading samples of the disagreements confirmed it: "best workout plans for women" and "best workout plan for women" were labelled different, while "count blank cells" and "count non blank cells" were labelled the same.

The cause is structural. Both benchmarks assign each query a *class* and call any two queries of a class a correct hit; neither judges the pair a cache actually meets.

In SearchQueries, classes are transitive closures of pairwise LLM judgements within embedding clusters. That is valid only if "same" is a strict equivalence, and three things break it. (1) A cached answer is written for its own query: a general query may be judged close enough to two specific ones, but the answer stored for one specific query is wrong for the other. (2) Tolerance accumulates along a chain of individually acceptable links. (3) One mistaken link merges two whole classes, labelling pairs no model ever compared.

In LMArena, a class is "generated from the same seed prompt". An LLM wrote the variants and nothing checked that a variant still asks the same thing; some reverse a premise or change an exam code or a number. In the other direction, identical prompts that came from different seeds are labelled different (37 such pairs in our sample).

Correction. We label the pair directly: the new query and the cached query it would be served from. A written guideline, fixed in advance, defines the same request strictly

Table 4. Cache guard on 2,000 test queries per dataset. Acc. = share of correct decisions; WR = wrong-reuse rate (a wrong cached answer was served), both in %. Thresholds are tuned for accuracy on dev. $ and p50 latency are per decision, averaged over the three datasets. vCache runs unmodified as a whole system: it learns its thresholds online from the benchmark's original class labels and caches only the prompts it missed, so its row is a reference point and not a like-for-like decider.

	GSM-Plus		LMArena		SearchQueries			
Decider	Acc.	WR	Acc.	WR	Acc.	WR	$/decision	p50
Jev	93.3	1.1	96.3	1.2	85.6	3.8	2.1×10^{-5}	0.18 s
Threshold → Jev	93.3	1.1	97.4	2.5	85.7	3.8	$\leq 2.3 \times 10^{-5}$	–
GPT-5.6	96.0	0.9	91.2	0.4	85.2	1.5	1.0×10^{-3}	1.25 s
GPT-OSS-20B	95.0	1.3	93.3	1.0	83.6	3.4	4.1×10^{-5}	1.31 s
Kev-4B (local)	88.1	5.3	96.9	1.9	85.4	8.6	local	0.07 s
Qwen3-8B (local)	86.3	2.5	95.7	1.8	84.8	6.5	local	0.39 s
Trained classifier	81.7	2.9	95.7	2.5	77.1	12.6	local	–
Similarity threshold	66.1	0.3	96.9	2.9	81.2	14.4	free	–
Cross-encoder	66.3	0.3	96.5	3.3	78.4	17.1	local	–
vCache ($\delta = 0.05$)	66.1	0.7	54.6	0.3	50.3	1.9	free	–

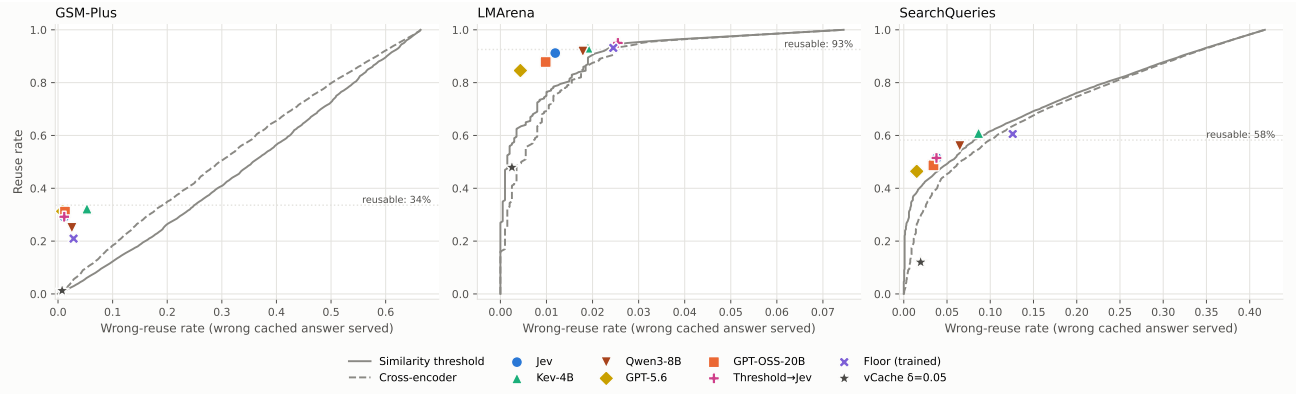


Figure 2. Reuse rate against wrong-reuse rate. Lines sweep the similarity threshold and the cross-encoder score; each decider is one point; the dotted line is the share of queries for which reuse is correct. Up and to the left is better. On GSM-Plus every learned decider sits far above both curves.

(rewording is the same; any change of entity, number, direction, task, scope or output constraint is different). A frontier LLM (Claude Opus 5.5) judged all 5,200 dev and test pairs blind, seeing neither the original label nor the similarity nor any decider's output. Pairs where a confident verdict disagreed with the original label were judged again, blind, by a separate instance, and a label changed only when both agreed with high confidence. This changed 156 of 2,600 LMArena labels (6.0%) and 409 of 2,600 SearchQueries labels (15.7%). On the corrected labels SearchQueries accuracy rose by 11 to 14 points for every decider, and the similarity threshold improved as well.

Limits. The corrector is an LLM, not a person, so the corrected labels may sit closer to an LLM decider's idea of the same request than human labels would. We reviewed every pair, not only disagreements, so that the correction could not steer toward any decider, and a human check of a random sample of changed and unchanged pairs is in progress. GSM-Plus labels are exact numeric answers and were not touched. We release both label sets and the reasons for every change.

6 Router

Task. Send a prompt to a small model or a large one. As with the cache guard, the decision replaces a generation: the expensive call is skipped when the small model would have been right.

Data. RouterBench (Hu et al., 2024) records graded answers and dollar costs of eleven models on the same prompts, so quality and model cost need no new calls. We use the pairing of the routing literature (Ong et al., 2024), Mixtral-8x7B as the small model and GPT-4 as the large one, on the six exactly graded task families, split by family into 1,998 test and 594 dev prompts, with the remaining 32,106 for training. The small model scores 0.589 on the test set and the large

Table 5. Routing on 1,998 RouterBench prompts. "Own choice" = share sent to the large model when the decider simply answers. Gain = quality over a random mix sending the same share, at the dev-tuned operating point (about 30% sent to the large model), in points.

Router	APGR [95% CI]	Own choice	Gain
Trained classifier	0.649 [.621, .678]	3.9%	+4.5
Jev	0.609 [.585, .642]	0.0%	+2.7
Kev-4B	0.582 [.553, .609]	0.0%	+3.0
GPT-OSS-20B	0.546 [.516, .576]	4.7%	+1.8
Qwen3-8B	0.531 [.488, .540]	20.6%	−1.1
GPT-5.6	0.529 [.510, .568]	2.3%	+1.2
Random mix	0.500	–	0

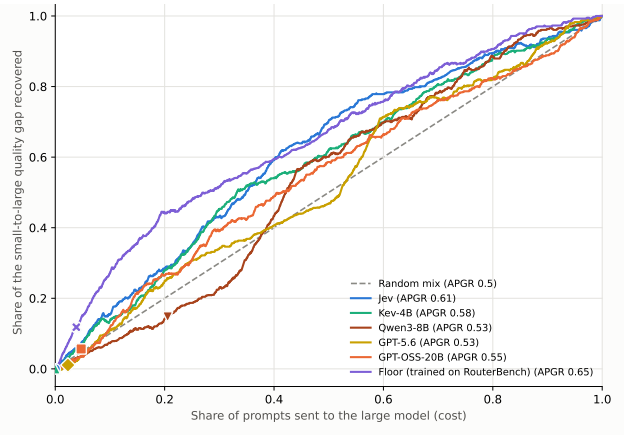


Figure 3. Share of the small-to-large quality gap recovered against the share of prompts sent to the large model. The dashed diagonal is a random mix. Markers show each decider's own choice.

one 0.792; 28% of prompts need the large model.

Deciders. Each decider sees the prompt only, is told which two models it is choosing between, and returns small or large with a probability. The trained baseline is a DistilRoBERTa classifier (Sanh et al., 2019) fine-tuned on the training split to predict whether the small model is wrong and the large one right.

Metrics. A decider's own choice gives one cost-quality point. Sweeping a threshold on its probability gives a curve, summarized by APGR (Ong et al., 2024): the share of the small-to-large quality gap recovered, averaged over all shares of prompts sent to the large model. A random mix scores 0.5. We also report one operating point fixed in advance: the threshold at the 70th percentile of each router's dev scores, applied unchanged to test.

Results. Table 5 and Figure 3 show three things.

First, asked directly, no decider routes. Jev and Kev send nothing to the large model, and GPT-5.6 sends 2.3%. At those points, quality is within 1.6 points of a random mix sending the same share.

Second, as rankers the general deciders carry a weak signal, and the frontier model carries the least. Jev's probabilities rank prompts best among the untrained deciders (APGR 0.609), ahead of GPT-5.6 by 0.080 [0.045, 0.106].

Third, the trained classifier beats every general decider, Jev included, by a small margin: +0.040 APGR [+0.001, +0.070]. At the fixed operating point it gains 4.5 points of quality over a random mix [3.3, 5.8], against 2.7 for Jev [1.5, 3.9]. Even so, it recovers little of the gap, which matches the routing literature's experience that learned routers are hard to separate from a random mix.

Why. The routing question is "will *this* small model get *this* prompt right?". The answer depends on that model's particular strengths, which the prompt does not show and a general decider has no way to know. A classifier trained on the model's record has seen that information; a decision model reading the prompt has not.

Takeaway. Avoiding a generation is not sufficient. The decider must also be able to see what the decision depends on.

7 Discussion

Two conditions. Across the three decision points, a standalone decider paid for its call only when both of the following held.

The decision avoids a generation. In agent control a generator runs after every decision, so the cheapest design lets that generator decide. At the cache guard and the router, one outcome of the decision skips the expensive call.

The evidence is in the input. Whether two queries ask the same thing can be read off the two queries. Whether a particular small model will answer a prompt correctly cannot be read off the prompt.

The cache guard meets both conditions and is where the decision model wins. Agent control fails the first and routing the second. We expect the conditions to transfer to other decision points: input guardrails and intent classification meet both, while "should the agent retry?" sits in front of a generation and likely does not.

The decision must be much cheaper than what it saves. A frontier LLM was the most accurate guard on GSM-Plus and still raised total cost above running without a cache. The relevant quantity is the price of a decision relative to the price of the generation it may avoid, weighted by how often it avoids one.

What the vendor claims amount to. The price claim holds:

Jev cost 22 to 48 times less per decision than the frontier model in our runs, and its latency was a steady 0.18 s. The quality claim holds at the cache guard and partly for single agent decisions (7 points below the frontier model). The implied claim that it can replace an LLM at any decision does not hold: in the agent loop and at the router, a cheaper or better alternative existed. Its probabilities were the best calibrated of any decider we measured, which is what makes cascades around it work.

Open decision models. Kev shows that the paradigm is reproducible but not uniformly: it matched Jev on judging whether two queries are the same request and collapsed on multi-step airline policy. On equal hardware its energy advantage over a same-size generative LLM making a constrained choice was about 20%, not an order of magnitude. A constrained LLM call emits very few tokens, so most of the work in both cases is reading the input.

Limitations. Jev is proprietary and versioned; our numbers describe version 1.13 through one gateway in one period. Agent control uses one domain, with labels and a generator from one frontier model. The corrected cache-guard labels come from an LLM, with human verification still in progress. Hosted deciders have dollar and latency measurements but no energy. RouterBench's models are from 2023; the router predicts those models' recorded correctness, so the structure of the result should hold, but the size of the gap may differ for current models. We did not fine-tune Kev or any LLM for a decision point; a tuned decision model might close the routing gap, at which point it is a trained classifier with a different architecture.

8 Related Work

Model routing and cascades. FrugalGPT (Chen et al., 2023) cascades models by a learned scorer; Hybrid LLM (Ding et al., 2024) and RouteLLM (Ong et al., 2024) train routers on quality labels; AutoMix (Aggarwal et al., 2023) lets a small model verify itself. RouterBench (Hu et al., 2024) provides the recorded outputs we route over. We add general, untrained deciders to this comparison and find that none replaces a router trained on the target model's record.

Semantic caching. GPTCache (Bang, 2023) reuses answers above a similarity threshold. vCache (Schroeder et al., 2025) learns per-prompt thresholds with an error bound and introduced the two traffic benchmarks we use. We evaluate a judge that reads both queries in place of a similarity score, and we show that the benchmarks' class-based labels are unreliable for pairwise decisions.

Small models for agent decisions. TinyAgent (Erdogan et al., 2024) and related work move tool selection to a small model with a larger model for execution. Our in-loop arms test this split with a frontier generator on τ^2-bench (Yao et al., 2024; Barres et al., 2025) and find that it does not beat a single call.

Decision models. Jev (TypeSafe AI, 2026) and Kev (Palmer, 2026) are recent; to our knowledge this is the first independent measurement of either across decision points with dollar, latency and energy costs.

9 Conclusion

We measured a non-generative decision model against LLMs and trained baselines at three decision points of an LLM system. It is a strong and very cheap semantic-cache guard, a poor trade inside an agent loop that must generate anyway, and no substitute for a trained router. The rule we draw is simple to apply before building: add a standalone decider where its decision can skip a generation and the input contains what the decision depends on, and make sure the decision costs far less than the generation it may save. The harness, all run records, the open baselines and the corrected cache-benchmark labels are available in an anonymized repository for review.

References

Aggarwal, P., Madaan, A., Anand, A., Potharaju, S. P., Mishra, S., Zhou, P., Gupta, A., Rajagopal, D., Kappaganthu, K., Yang, Y., Upadhyay, S., Faruqui, M., and Mausam. AutoMix: Automatically mixing language models. *arXiv preprint arXiv:2310.12963*, 2023.

Bang, F. GPTCache: An open-source semantic cache for LLM applications enabling faster answers and cost savings. In *Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS)*, 2023.

Barres, V., Dong, H., Ray, S., Si, X., and Narasimhan, K. τ^2-bench: Evaluating conversational agents in a dual-control environment. *arXiv preprint arXiv:2506.07982*, 2025.

Chen, L., Zaharia, M., and Zou, J. FrugalGPT: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023.

Chiang, W.-L., Zheng, L., Sheng, Y., Angelopoulos, A. N., Li, T., Li, D., Zhang, H., Zhu, B., Jordan, M., Gonzalez, J. E., and Stoica, I. Chatbot arena: An open platform for evaluating LLMs by human preference. In *International Conference on Machine Learning*, 2024.

Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., and Schulman, J. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

Craswell, N., Campos, D., Mitra, B., Yilmaz, E., and Billerbeck, B. ORCAS: 18 million clicked query-document pairs for analyzing search. In *Proceedings of the 29th ACM International Conference on Information and Knowledge Management*, 2020.

Ding, D., Mallick, A., Wang, C., Sim, R., Mukherjee, S., Rühle, V., Lakshmanan, L. V. S., and Awadallah, A. H. Hybrid LLM: Cost-efficient and quality-aware query routing. In *International Conference on Learning Representations*, 2024.

Erdogan, L. E., Lee, N., Jha, S., Kim, S., Tabrizi, R., Moon, S., Hooper, C., Anumanchipalli, G., Keutzer, K., and Gholami, A. TinyAgent: Function calling at the edge. *arXiv preprint arXiv:2409.00608*, 2024.

Guo, C., Pleiss, G., Sun, Y., and Weinberger, K. Q. On calibration of modern neural networks. In *International Conference on Machine Learning*, 2017.

Hu, Q. J., Bieker, J., Li, X., Jiang, N., Keigwin, B., Ranganath, G., Keutzer, K., and Upadhyay, S. K. RouterBench: A benchmark for multi-LLM routing system. *arXiv preprint arXiv:2403.12031*, 2024.

Kahneman, D. *Thinking, Fast and Slow*. Farrar, Straus and Giroux, 2011.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023.

Li, Q., Cui, L., Zhao, X., Kong, L., and Bi, W. GSM-Plus: A comprehensive benchmark for evaluating the robustness of LLMs as mathematical problem solvers. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024.

Li, Z., Zhang, X., Zhang, Y., Long, D., Xie, P., and Zhang, M. Towards general text embeddings with multi-stage contrastive learning. *arXiv preprint arXiv:2308.03281*, 2023.

Ong, I., Almahairi, A., Wu, V., Chiang, W.-L., Wu, T., Gonzalez, J. E., Kadous, M. W., and Stoica, I. RouteLLM: Learning to route LLMs with preference data. *arXiv preprint arXiv:2406.18665*, 2024.

Palmer, J. Kev: an open-weights decision model. https://github.com/jaredpalmer/kev, 2026. Commit 557598f.

Reimers, N. and Gurevych, I. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 2019.

Sanh, V., Debut, L., Chaumond, J., and Wolf, T. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.

Schroeder, L. G., Desai, A., Cuadron, A., Chu, K., Liu, S., Zhao, M., Krusche, S., Kemper, A., Zaharia, M., and Gonzalez, J. E. vCache: Verified semantic prompt caching. *arXiv preprint arXiv:2502.03771*, 2025.

TypeSafe AI. Jev: a System One decision model. https://docs.typesafe.ai, 2026. Version 1.13.

Yang, A. et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Yao, S., Shinn, N., Razavi, P., and Narasimhan, K. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.

A Additional Results

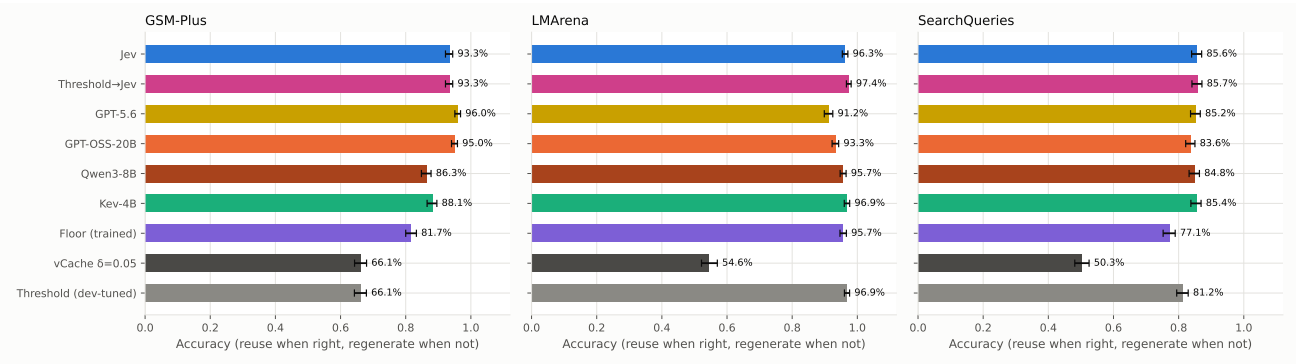


Figure 4. Cache-guard accuracy per decider and dataset with 95% CIs clustered by class. vCache runs as a whole system on the benchmark's original class labels (see Table 4).